

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Fluxia - Gestión de Provisiones</title>
  <link href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:wght@400;500;600;700&family=IBM+Plex+Mono:wght@400;600&display=swap" rel="stylesheet">
  <style>
    :root {
      --bg-app: #f1f5f9;
      --card-bg: #ffffff;
      --text-main: #0f172a;
      --text-muted: #64748b;
      --primary: #0d9488;
      --primary-hover: #0f766e;
      --yellow-bg: #fef3c7;
      --yellow-border: #fde68a;
      --yellow-text: #92400e;
      --border-color: #e2e8f0;
    }

    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
      font-family: 'IBM Plex Sans', sans-serif;
    }

    body {
      background-color: var(--bg-app);
      color: var(--text-main);
      padding: 16px;
      display: flex;
      justify-content: center;
    }

    .mobile-container {
      width: 100%;
```

```
    max-width: 420px;
    background: var(--bg-app);
    display: flex;
    flex-direction: column;
    gap: 16px;
  }
```

```
.btn-registrar {
  width: 100%;
  background-color: var(--primary);
  color: white;
  border: none;
  padding: 14px;
  border-radius: 12px;
  font-size: 16px;
  font-weight: 600;
  cursor: pointer;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
}
```

```
.btn-registrar:hover {
  background-color: var(--primary-hover);
}
```

```
.summary-banner {
  background: var(--card-bg);
  padding: 12px 16px;
  border-radius: 12px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  font-size: 14px;
  font-weight: 500;
  box-shadow: 0 1px 3px rgba(0,0,0,0.05);
}
```

```
.summary-amount {
  font-family: 'IBM Plex Mono', monospace;
  font-weight: 600;
}
```

```
.pending-container {  
  display: flex;  
  flex-direction: column;  
  gap: 10px;  
}
```

```
.yellow-card {  
  background-color: var(--yellow-bg);  
  border: 1px solid var(--yellow-border);  
  border-radius: 12px;  
  padding: 14px 16px;  
  display: flex;  
  flex-direction: column;  
  gap: 10px;  
  box-shadow: 0 2px 4px rgba(0,0,0,0.02);  
}
```

```
.yellow-card-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  font-weight: 700;  
  font-size: 15px;  
  color: var(--yellow-text);  
}
```

```
.yellow-card-amount {  
  font-family: 'IBM Plex Mono', monospace;  
  font-size: 16px;  
}
```

```
.yellow-card-desc {  
  font-size: 13px;  
  color: #78350f;  
}
```

```
.yellow-card-actions {  
  display: flex;  
  gap: 8px;  
  justify-content: flex-end;  
  flex-wrap: wrap;
```

```
}
```

```
.card-btn {  
  padding: 6px 12px;  
  border-radius: 6px;  
  font-size: 12px;  
  font-weight: 600;  
  border: none;  
  cursor: pointer;  
}
```

```
.btn-reponer-action {  
  background-color: var(--primary);  
  color: white;  
}
```

```
.btn-edit-action {  
  background-color: #d97706;  
  color: white;  
}
```

```
.btn-yno-action {  
  background-color: #475569;  
  color: white;  
}
```

```
.section-title {  
  font-size: 12px;  
  font-weight: 700;  
  color: var(--text-muted);  
  letter-spacing: 0.5px;  
  margin-top: 12px;  
  margin-bottom: 4px;  
  text-transform: uppercase;  
}
```

```
.history-list {  
  display: flex;  
  flex-direction: column;  
  gap: 8px;  
}
```

```
.history-item {
  background: var(--card-bg);
  padding: 12px 16px;
  border-radius: 10px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  border: 1px solid var(--border-color);
}

.history-info {
  display: flex;
  flex-direction: column;
  gap: 2px;
}

.history-title {
  font-size: 14px;
  font-weight: 600;
}

.history-date {
  font-size: 12px;
  color: var(--text-muted);
}

.history-amount {
  font-family: 'IBM Plex Mono', monospace;
  font-weight: 600;
  color: #ef4444;
}
</style>
</head>
<body>

<div class="mobile-container">
  <button class="btn-registrar" onclick="registrarNuevoUso()">+ Registrar uso</button>

  <div class="summary-banner">
    <span>⌚ Pendiente de reponer a tus provisiones</span>
```

```
    <span class="summary-amount" id="total-pending">0,00 €</span>
  </div>
```

```
<div class="pending-container" id="pending-cards-container"></div>
```

```
<div class="section-title">Usos registrados · Historial</div>
```

```
<div class="history-list" id="history-list-container"></div>
```

```
</div>
```

```
<script>
```

```
  // Datos por defecto si es la primera vez que entra
  const defaultData = [
    { id: 1, tipo: "RESCATE", concepto: "Tributos (IBI, Basura, Vehículo)", fecha:
"2026-09-10", importe: 390.48, pendienteReponer: 390.48 },
    { id: 2, tipo: "RESCATE", concepto: "Boda Ana", fecha: "2026-09-17", importe: 100.00,
pendienteReponer: 100.00 },
    { id: 3, tipo: "RESCATE", concepto: "Seguro de coche", fecha: "2026-09-20", importe:
45.00, pendienteReponer: 45.00 },
    { id: 4, tipo: "Retiro temporal", concepto: "Mantenimiento césped", fecha: "2026-09-20",
importe: 35.00, pendienteReponer: 35.00 }
  ];

  // Cargar desde localStorage o usar los datos por defecto para no perder nada
  let usosData = JSON.parse(localStorage.getItem('fluxia_usos')) || defaultData;

  function guardarDatos() {
    localStorage.setItem('fluxia_usos', JSON.stringify(usosData));
  }
```

```
function renderApp() {
  const pendingContainer = document.getElementById('pending-cards-container');
  const historyContainer = document.getElementById('history-list-container');
  const totalPendingElement = document.getElementById('total-pending');
```

```
  pendingContainer.innerHTML = "";
  historyContainer.innerHTML = "";
```

```
  let totalPendingSum = 0;
```

```
  usosData.forEach(item => {
    if (item.pendienteReponer > 0) {
```

```

totalPendingSum += item.pendienteReponer;

const card = document.createElement('div');
card.className = 'yellow-card';
card.innerHTML = `
  <div class="yellow-card-header">
    <span>🔄 ${item.tipo.toUpperCase()}</span>
    <span class="yellow-card-amount">${
{item.pendienteReponer.toFixed(2).replace('.', ',')} €</span>
  </div>
  <div class="yellow-card-desc">De «${item.concepto}» · ${item.fecha}</div>
  <div class="yellow-card-actions">
    <button class="card-btn btn-reponer-action" onclick="reponerItem($
{item.id})">Reponer</button>
    <button class="card-btn btn-edit-action" onclick="editarItem($
{item.id})">Editar</button>
    <button class="card-btn btn-yno-action" onclick="marcarYaNo($
{item.id})">Archivar</button>
  </div>
`;
pendingContainer.appendChild(card);
} else {
const historyItem = document.createElement('div');
historyItem.className = 'history-item';
historyItem.innerHTML = `
  <div class="history-info">
    <span class="history-title">${item.concepto} · ${item.tipo}</span>
    <span class="history-date">${item.fecha} · Repuesto / Completado</span>
  </div>
  <div style="display: flex; align-items: center; gap: 10px;">
    <span class="history-amount">-${item.importe.toFixed(2).replace('.', ',')} €</
span>
    <button class="card-btn btn-edit-action" onclick="editarItem(${item.id})"
style="padding: 4px 8px; font-size: 10px;">Editar</button>
  </div>
`;
historyContainer.appendChild(historyItem);
}
});

totalPendingElement.textContent = totalPendingSum.toFixed(2).replace('.', ',') + ' €';

```

```

    guardarDatos();
}

function reponerItem(id) {
    const item = usosData.find(u => u.id === id);
    if (item) {
        item.pendienteReponer = 0;
        renderApp();
    }
}

function marcarYaNo(id) {
    const item = usosData.find(u => u.id === id);
    if (item) {
        // En lugar de borrarlo por completo, lo dejamos en 0 de pendiente para que baje al
historial
        item.pendienteReponer = 0;
        renderApp();
    }
}

function editarItem(id) {
    const item = usosData.find(u => u.id === id);
    if (!item) return;

    const nuevoConcepto = prompt("Modifica el concepto:", item.concepto);
    if (nuevoConcepto === null) return; // Si cancela

    const nuevoImporteStr = prompt("Modifica el importe (€):", item.importe);
    if (nuevoImporteStr === null) return;

    const nuevoImporte = parseFloat(nuevoImporteStr.replace(',', '.'));
    if (isNaN(nuevoImporte) || nuevoImporte <= 0) {
        alert("Por favor, introduce un importe válido.");
        return;
    }

    item.concepto = nuevoConcepto;
    item.importe = nuevoImporte;
    // Si estaba pendiente, actualizamos también el pendiente con el nuevo importe
    if (item.pendienteReponer > 0) {

```

```
        item.pendienteReponer = nuevImporte;
    }

    renderApp();
}

function registrarNuevoUso() {
    const concepto = prompt("Concepto del uso o retiro:");
    if (!concepto) return;

    const importeStr = prompt("Importe (€):");
    const importe = parseFloat(importeStr ? importeStr.replace(',', '.') : 0);
    if (isNaN(importe) || importe <= 0) {
        alert("Importe no válido.");
        return;
    }

    const tipo = prompt("Tipo (escribe 'Rescate' o 'Retiro temporal'):", "Retiro temporal");

    const nuevold = usosData.length > 0 ? Math.max(...usosData.map(u => u.id)) + 1 : 1;
    const fechaHoy = new Date().toLocaleString().split('T')[0];

    usosData.unshift({
        id: nuevold,
        tipo: tipo || "Retiro temporal",
        concepto: concepto,
        fecha: fechaHoy,
        importe: importe,
        pendienteReponer: importe
    });

    renderApp();
}

document.addEventListener('DOMContentLoaded', renderApp);
</script>
</body>
</html>
```